

Design of Integrated Manufacturing System Control Software

ARCH W. NAYLOR, SENIOR MEMBER, IEEE, AND RICHARD A. VOLZ, SENIOR MEMBER, IEEE

Abstract—A coordinated multifaceted conceptual framework for the development of real-time control software for integrated manufacturing systems is described. Typically, such software is developed using outmoded software tools and is invariably tailored to each manufacturing system. Consequently, it is expensive to develop and maintain, inflexible, and relatively unreliable. One reason, though certainly not the only one, so few integrated manufacturing systems exist is that control software has been created in this flawed way. A more orderly, rational, and structured approach is obviously needed, and one is described here. The approach is based on a blending of modern software concepts and formal semantic models. The software concepts are 1) the use of software components (extended to include hardware), 2) a common distributed language environment, and 3) generic (i.e., reusable) software. Control software is constructed as an assemblage of components written in the common distributed language. The formal models model the semantics of the assemblages. Generics are used to enhance the reusability of software. The approach was tested using a network of VAX computers as a testbed.

I. INTRODUCTION

ARGUABLY, software is the integrated manufacturing problem [1]. The machines, robots, material transports, and so forth exist, but the software needed to tie them together into orchestrated flexible robust systems does not. The software for the relatively few integrated systems that do exist is very application specific, difficult to maintain, difficult to change, difficult to port to other hardware, not robust, and expensive. The following list of typical device characteristics illustrates some of the problems:

- *Numerically controlled (NC) machines*—programmed in APT-like languages [2]; complex special-purpose controllers with only a limited interface to the outside world;
- *robots*—programmed in VAL [3], Karel, or AML-like languages [4], [5], or, even more likely, by teaching; controllers built upon general-purpose computers, but interfaces to the outside world often limited;
- *programmable controllers*—programmed in ladder logic, limited other programming capabilities; usually

built out of special-purpose hardware; only limited interfaces to other computers;

- *materials handling and storage/retrieval systems*—typically controlled by a general-purpose computer using languages such as assembly, Fortran, Pascal, or C.

Thus controllers are not designed for interconnection; there are too many languages, many of them inadequate and out of date, while others are special-purpose; controller hardware often lacks the functionality of general-purpose computers; there is almost no use of modern software techniques; and there is no overall theoretical base for integrating the operation of such systems.

A basic need exists to be able to connect things together physically—that is, a need for *physical* and *low-level* interconnection standards—and this has been recognized. MAP [6] and the work at the National Bureau of Standards [7] are outstanding examples of the progress that is being made, and consequently, interconnection, at least, should cease to be a major hurdle in the near future.

However, as important as low-level interconnection standards are, their existence still leaves major unsolved software problems. Some of these are simply characteristic of large software projects; others are problems of distributed computing; still others arise from the special nature of manufacturing systems themselves. Taken together, they define the next important hurdle for integrated manufacturing systems. How should the large complex distributed software systems needed in integrated manufacturing systems be designed, implemented, and maintained? This paper describes an approach for answering some of these questions.

In particular, we develop a new conceptual framework for integrated manufacturing system control software. Examples of other conceptual frameworks, for nonmanufacturing systems, are the relational database model [8], the ISO network model [9], and the algebraic model for abstract data types [10]. Often they presume, as will ours, a design style. For example, the relational conceptual framework presumes relational databases and effectively excludes hierarchical and network databases. Mainly, conceptual frameworks allow one to describe a situation of interest precisely, to pose problems clearly, and to formulate solution methods. The need for a conceptual framework for manufacturing control software was a major conclusion of the recent NSF Workshop on Manufacturing Systems Integration [11].

Manuscript received April 11, 1987; revised July 20, 1987.

This work was partially presented at the National Academy of Engineering Meeting, February 1987.

A. W. Naylor is with the University of Michigan, 3118 Electrical Engineering and Computer Science Building, 1301 Beal, Ann Arbor, MI 48109-2122.

R. A. Volz is with the Robots Systems Division, University of Michigan, 110 Advanced Technology Laboratories Building, 1101 Beal, Ann Arbor, MI 48109-2110.

IEEE Log Number 8716926.

0018-9472/87/1100-0881\$01.00 ©1987 IEEE

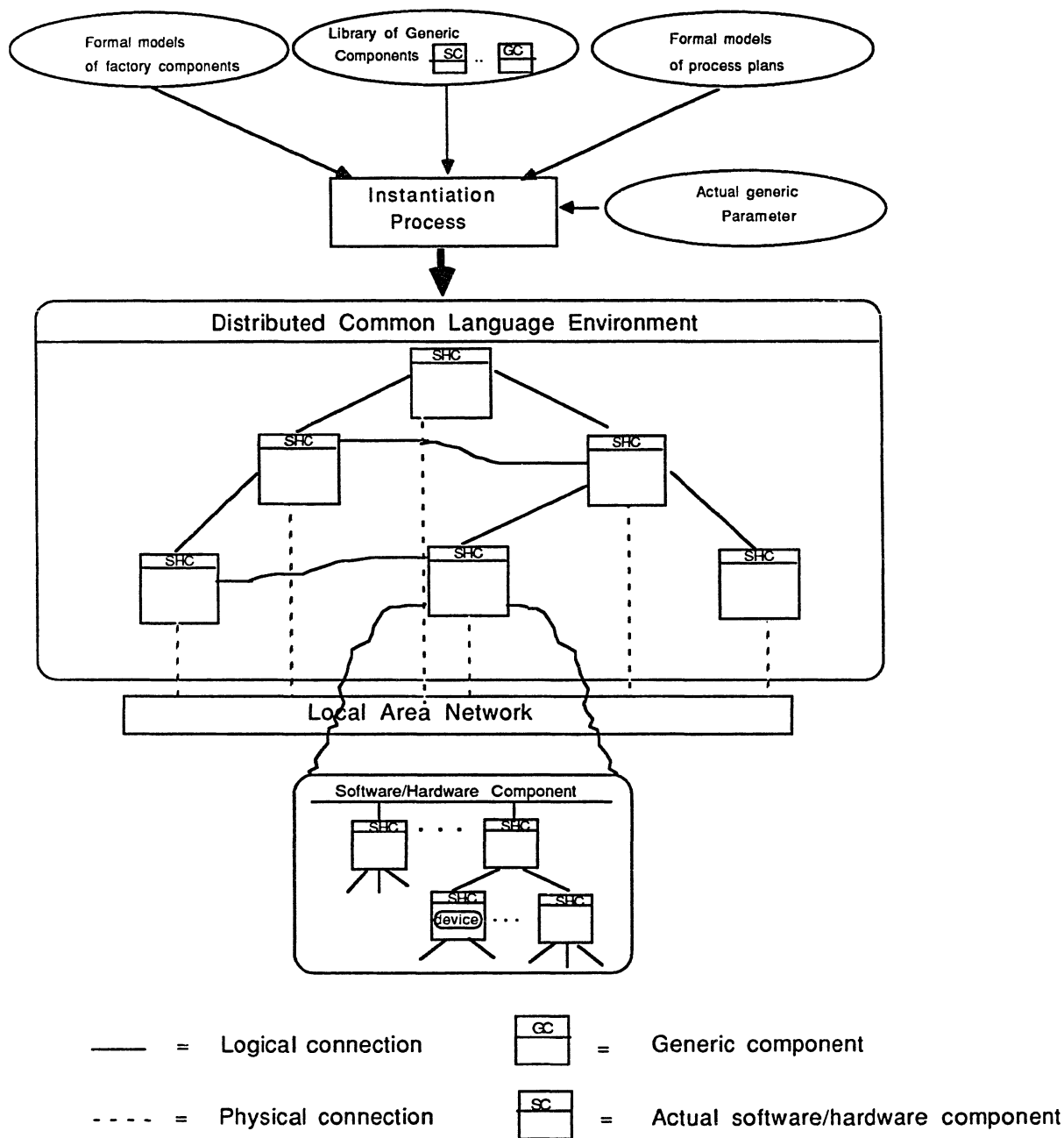


Fig. 1.

We propose a conceptual framework based on an intimate blending of modern software concepts and formal models. In particular, we view an integrated manufacturing system as an assemblage of software/hardware components, where "software/hardware component" is a generalization of the software component concept in which, for instance, the internals of a component instead of software might be a robot [12]. The formal models describe the semantics of these components and their assemblages.

Ideally, we want these components and assemblages to be generic in a way that allows them to be used in different

manufacturing systems, and we would like the creation of assemblages and their models to be computer-aided, semi-automatic, or perhaps even automatic. Further, the assemblages are distributed; therefore, issues of distributed processing must be considered and incorporated into the conceptual framework.

Finally, we argue that the foregoing requires a common language environment. However, that does not mean a monolithic software, nor does it rule out certain parts being written in languages other than the common language. By "common language environment" we mean that

the software providing overall structure, and probably most other parts as well, is written in a common language. Software in other languages will frequently be associated with particular devices—for example, a robot programmed in VAL—and can usually be encapsulated into shells crafted from the common language.

Section II provides a brief overview of our conceptual framework. Section III describes what we call “software/hardware components.” Section IV presents our view of generics—actually a generalized idea of generics—and the use of generics in manufacturing software. Section V presents the modeling formalism that we use for modeling semantics. Section VI discusses the role of distributed languages. Section VII discusses briefly an experimental implementation of these ideas.

II. OVERVIEW OF CONCEPTUAL FRAMEWORK

Our conceptual framework for manufacturing software is based upon the following key concepts that are illustrated in Fig. 1:

- recursively defined hardware/software components encompassing both the logical and physical aspects of the entity being defined;
- formal semantic models of the components and their assemblages, and of process plans;
- generic components that can be instantiated into real components by supplying appropriate parameters (either statically at the time a system is built, or during an initialization period with parameters supplied by physical devices attached to the system);
- libraries of components; and
- a common distributed language environment.

Our view of hardware/software components is both recursive and distributed. That is, components may be built up out of other components and need not all reside on the same computer. In fact, one component may itself span several computers. Each physical device on the factory floor is encapsulated inside a software/hardware component and can be thought of as having two faces. One is made up of its interactions on the factory floor. The other is the software interface connected to the higher levels of the manufacturing software system. Our components must encompass both; indeed, the purpose of the software is to control the interactions on the factory floor through the software interface.

The formal modeling part of our framework is expressed in a simple extension to first-order logic [13] that can be used to represent the process plans that the cell is to implement as well as the actions of the components. The uniform modeling of process plans and software/hardware components simplifies the software structure and allows one to view the process plan as just another component in the system.

The generic components are abstractions of real components designed to allow reusability of component software. By *instantiating* them with actual parameters, real compo-

nents are created and can be used as part of a real factory control system. The top half of Fig. 1 illustrates the relationship among the models, generic components and actual parameters. The instantiation process shown here is a generalization of the simple generic instantiation process of languages like Ada.³

One component may include models of other components. The models may then be used in a predictive simulation manner to examine the likely outcome of a possible control strategy before it is actually applied. Further, the formal models of process plans can be converted to actual components that drive the operation of a system. At present the translation from the formal models to actual software is performed manually, but conceptually (at present, and in the future actually) they could be converted automatically. These instantiated components are then used in the control software being developed, as shown in the bottom half of Fig. 1.

The bottom half of Fig. 1 illustrates not only a manufacturing software system, but the environments in which it was created and operates. While the underlying digital communications network is shown in this figure, it is logically contained within the distributed language environment. That is, the software for controlling the manufacturing system is written using normal interprocess communication mechanisms within the language, without any explicit references to interprocessor communication. There is a mapping that assigns program parts to the different processors in the system. The language translation system ensures that interprocess communication is appropriately translated to interprocessor communication when necessary. Thus the programmer does not have to be concerned with network protocols or inventing application level inter-program protocols.

III. SOFTWARE/HARDWARE COMPONENTS

The main ideas of this section are that software/hardware components and their assemblages offer a powerful way of dealing with manufacturing control software, that their use is best done in a common language environment, and that a common language environment is an ideal environment within which to carry out system integration.

A. Software Component Concept

Writing modular programs is an old idea, and the term “software component” is—almost—another name for a module or subroutine. However, there are very important distinctions. Modular programming is largely a programming style, while, as we mean it, “software component” is an enforced language concept. Specifically, our software components have three basic characteristics: 1) a well-defined public interface, 2) an internal implementation that is inaccessible to the user (see Fig. 2), and 3) both the visible part and the inaccessible implementation of software components should be separately compilable from

³Ada is a registered service mark of the Ada Joint Program Office.

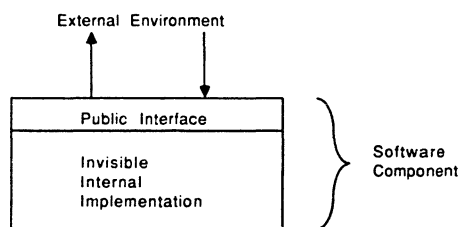


Fig. 2.

the program components that use them, and it should be possible to compile the inaccessible and visible parts separately. The combination of a public interface and hidden internals 1) provides a clear understanding of how the software component is to be used and 2) precludes access to the internals of the implementation. The latter is crucial for preventing large classes of programming errors, allowing extensive compiler error checking, decreasing debugging time, and increasing program reliability. Separate compilation is important because it allows the development of "plug compatible" software modules and a software components industry, as we explain later.

While the most commonly used languages for manufacturing software, e.g., Fortran, Cobol, Pascal, and Basic, do not provide the needed support for software components, what is needed in a language that would support them is well understood in the software engineering community [14]. Languages do exist that meet many of these needs: Modula-2 [15], Ada [16], C++ [17], NIL [18], and CSP [19] are but a few examples, with Modula-2 and Ada probably going furthest.

To illustrate the idea of a software component, we will use the Ada package construct, as Ada is the most widespread of these languages. In Ada, the public interface is called the "specification" and the internal implementation, the "body." A stack is a classic, simple example. The specification might be as follows:

```
package STACK is
  procedure PUSH (X:INTEGER);
  function POP return INTEGER;
end STACK.
```

This, and an understanding of the operation (i.e., semantics) of stacks, are all the user of this software component needs to know. The body of the stack would be as follows:

```
package body STACK is
  ...
  ...
end STACK;
```

We have purposely not filled in the body to emphasize that it is not available to the user in the sense that it can only be accessed through the specification. Indeed, the user may not even have the source code for the body.

B. The Software/Hardware Component Concept

Basic to our conceptual framework is the extension of the concept of software components to include manufacturing subsystems [12]. To illustrate, the specification for a simple material transport vehicle might be as follows:

```
package VEHICLE is
  procedure MOVE_FORWARD;1
  procedure MOVE_BACKWARD;
  procedure STOP;
  function MOVING return BOOLEAN;
end VEHICLE;
```

In this case the body of the "software component" includes the actual vehicle, and to signal this we refer to it as a "software/hardware component."

In this straightforward mode of operation software components will be available from vendors. The purchaser will be given the compiled specification and body for the target machine and a source code listing for the specification. The user can make calls to the component by referencing only the specification in his or her program. Furthermore, given only the component's compiled specification, the user can compile his or her program. The body is simply linked into the rest of the program at link time. Since a body is compiled after its specification and since the body, having been purchased "off the shelf," does not depend on any other software component in the user's program, this can be easily accomplished. Thus the specification is a powerful high-level mechanism of integration that establishes an interconnection between the purchased software component and the rest of the user's program. Different vendors' compiled bodies could be linked in as long as their specifications matched, i.e., plug compatible software can become a reality. Of course, the user can still write his/her own software component.

Usually, software components—particularly when thought of as abstract data types—are passive in the sense that they only react to calls. In our case, however, we must leave open the possibility of component being active in that it may originate an event itself, without being called. For example, a contact sensor on a robot may raise an interrupt. There are several explicit ways to handle this asynchronous behavior. However, we will not go into detail here. The main point of all of them is that a name identifying the event must be exported in one way or another. In particular, it is important that one-way naming be preserved, that is, the software component cannot know the names of its users.

Our extended view of software components also allows an entirely new mode of interaction between manufacturers and suppliers of manufacturing equipment. Manufacturers (purchasers of manufacturing equipment) could design the software component specifications to provide

¹Although we show only procedures and functions in the examples of specifications in this article, one would also want to allow task entries in practice.

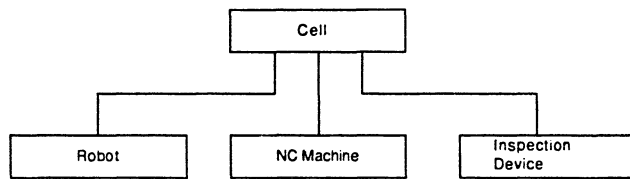


Fig. 3.

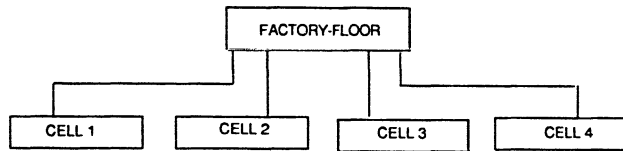


Fig. 4.

the view of the manufacturing device necessary for the application at hand. Manufacturing equipment suppliers would then be given the specifications and would have to provide not only the required hardware, but a body to the component package that is compatible with the purchaser's specification as well, which would "plug" into the rest of the system. Since the software component would now be carefully specified, several vendors might bid against each other for the job. This is exactly the opposite of current practice in which the purchaser assumes the responsibility for custom designing the software interfaces to vendor-supplied devices to integrate them into the system. It also places responsibility for development of the body of the software component with the people most familiar with the device that must be controlled.

C. Software/Hardware Components as Building Blocks

Robot, NC machine, and inspection device components, might be assembled to create a new software/hardware component called, say, CELL, as shown in Fig. 3. As described before, the robot, NC machine, and inspection device each have a public interface (specification) that is available to the internals (body) of the software/hardware component CELL. This component, in turn, will have a public interface that is made available to other parts of the system. For example, several cells might be assembled to form a component called FACTORY_FLOOR as shown in Fig. 4. In either case—CELL or FACTORY_FLOOR—we have "multiple inheritance," that is, the component CELL, for example, inherits, modifies, and, perhaps, adds to the functionality of the components, robot, NC machine, and inspection device.

There are two important points. First, the programmer of CELL merely refers to the public interfaces of robot, NC machine, and inspection device, and the language translation system takes care of the actual interconnections. Second, to use the component CELL, or any other one for that matter, we need to understand the interactions of the robot, NC machine, and inspection device as physical

devices on the factory floor. That is, we need to know the semantics of each of these components and the way they are set up with respect to one another on the factory floor. This is precisely what the formal semantics models described later give us.

D. Integration

We are suggesting a largely common distributed language environment, in particular, one in which the system integration is accomplished. That is, except as noted later, the software would be a single program written (essentially) in one language. This program would, however, be far from monolithic; it would be structured as an assemblage of software/hardware components. This approach offers at least three important advantages.

- It allows the programmers to think about the total software system without being artificially constrained by processor boundaries.
- The compiler can check for errors across the entire system, not just within the separate parts.
- The interprocessor communication aspects of integration can be made implicit. Since handling communications explicitly is typically a major programmer overhead task, this is an extremely important saving.

However, in cases where a piece of software written in another language must be used, it is usually possible to encapsulate a software product in a component whose specification is compatible with the rest of the system being developed. The problem is akin to the problem of building a device driver, except that in this case it is (perhaps) purely software that is being interfaced to. The problem reduces to building a single interface to the program being absorbed. The problem may not be easy and may require the assistance of the original program builder, but at least there is only a single point of interface to the larger system being built, and once complete, the component can be incorporated easily in many different systems. Also note that many pieces of software written in other languages may only need to be "executed" by the control software. For example, from the vantage point of the control software for the integrated system, a parts program is just something to execute. Thus control software written in the common language might be concerned with the uploading, downloading, starting, terminating, and so forth, of parts programs written in the other languages.

E. Factory Simulation

We are interested in simulations for at least two reasons. First, when developing and debugging control software, we would like to replace the actual factory floor with a simulation. Second, certain important control algorithms contain a simulation that is employed to investigate alternative future scenarios. Fortunately, in either case, creating software as an assemblage of software/hardware compo-

nents lends itself well to simulation. In particular, one can obtain a simulation by replacing the body of each factory floor component—NC machines, robots, and so forth—by software that simulates the semantics of the device. However, one must also simulate interactions of these devices. Doing so using our conceptual framework is easy. In particular, the devices and their interactions can be modeled using the modeling formalism discussed below, and this model can, in effect, be an assemblage of submodels that mirrors the assemblage of software/hardware components.

Further, this approach allows different kinds of simulation clocks to be employed. For example, a clock equal to real time is possible if we want a simulator that, when viewed through its interface, acts exactly like the actual factory floor. Or, if we want a simulator for investigating alternative future scenarios, we can have a clock similar to the one in GPSS and other discrete event simulation languages. Either clock is compatible with our approach to simulation. The foregoing approach to simulation is an extension of [20] in which formal descriptions of manufacturing cells were extended to include simulations.

IV. GENERICS

Generics are important because they amplify the concept of reusable software and, in our conceptual framework, support semi-automatic model building and programming. Roughly speaking, a generic software component is a template that can be instantiated to get an actual component. A stack is, here also, a classic example:

```
generic
  type ELEMENT is private;
package STACK is
  procedure PUSH (E: in ELEMENT);
  procedure POP (E: out ELEMENT);
end STACK.
```

The point of this generic stack is that it can be used to stack various types of elements. One merely supplies a data type for the generic parameter ELEMENT. For example,

```
package INTEGER_STACK is new STACK (INTEGER)
```

yields a stack for integers. If stacks for other items are needed, other instantiation of the generic package with different type parameters will create them. Usually, the generic package resides in a library, making it usable in more than one program.

We extend this concept to software/hardware components. Imagine a generic component for material transport vehicles. An instantiation would be a specific vehicle. Information characterizing this vehicle would be passed to the instantiation of the generic component through generic parameters, in the spirit of ELEMENT above, and become part of the real instance of the component, as shown in Fig. 5. An instantiation might be as follows:

```
package AGV_7 is new VEHICLE (VOLVO);
```

where VOLVO is an appropriately structured database that characterizes a particular VOLVO automatically guided vehicle. This looks similar to the earlier instantiation of STACK; however, there are differences. In this case we not only have to supply the information VOLVO, we must supply the actual vehicle. Things might occur as follows: various vehicle vendors would be given the generic interface specification for the generic vehicle component and be told the structure of the generic formal parameters, i.e., the structure that the database VOLVO satisfies. The vendor would supply the generic body (software portion of the internals) of the software/hardware component along with the vehicle.

Although the vendor-user scenario just described is obviously similar to that described for software/hardware components (Section III-B), the generic components and declared actual components² are different. Contrast the declaration of AGV_7_DEC as a component below with the above instantiation of the generic component VEHICLE:

```
package AGV_7_DEC is
  ...
end AGV_7_DEC;
```

AGV_7_DEC and the instantiation AGV_7 both refer to specific vehicles. However, we can use the software in the generic component VEHICLE to create actual components for many different vehicles, and these need not be of the same type. For example,

```
package AGV_7 is new VEHICLE (VOLVO);
package AGV_8 is new VEHICLE (VOLVO);
package AGV_9 is new VEHICLE (GM);
```

The package AGV_7_DEC is forever associated with a specific vehicle³ (or, perhaps, a simulation of it, as discussed in Section III-E).

Normally, a generic component is instantiated at compile time, but we allow “run-time instantiations” as well. By this, we mean supplying the information at run time. In this case the model of the instantiation looks like that of Fig. 6. Whether this should still come under the heading of “generics” and whether we should say “instantiation” is arguable. Perhaps “model reference” would be a preferable term. Be that as it may, we want to allow both cases: compile-time and run-time instantiation. For example, we might want the generic vehicle component to query the vehicle itself at run time to acquire the information needed for instantiation.

A far more powerful, and probably more readily achievable, use of generic components occurs in the case of assemblages of components. For example, imagine a generic cell component. It might presuppose that it will use

²By “declared actual components” we mean components that are not the result of instantiating a generic component, this is, these components are declared directly (see Fig. 1).

³However, it would be possible to associate the public interface of AGV_7_DEC with another vehicle by replacing AGV_7_DEC's body with that of another vehicle.

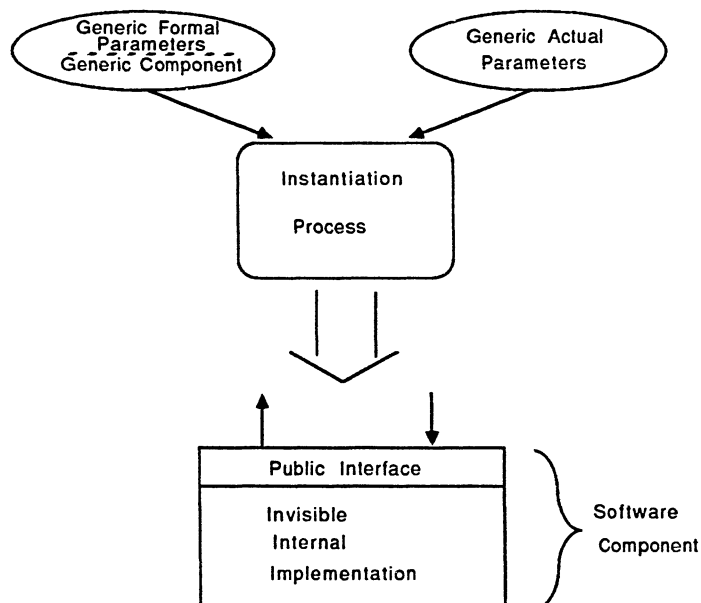


Fig. 5.

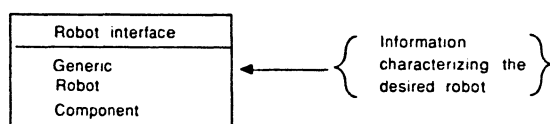


Fig. 6.

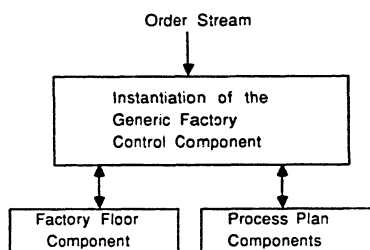


Fig. 7.

generic parameters—either at compile time or run time—would include a model of the factory floor to be controlled and descriptions of the process plans for the various parts to be made. These models would be formulated using the formal modeling system described later. The factory floor might itself be an instantiation of a generic factory floor component, and similarly, the process plans might be instantiations of a generic process plan component. Along the way, we might have used a generic cell control component. The situation would be as shown in Fig. 7. We have assumed that the interface to the control component is simply an input order stream. The control component sends commands to the factory floor component and receives status information, and the process plan component is used to keep track of where each part is in its process plan. Needless to say, we understand that a truly generic factory control component, that is, one for an arbitrary factory making an arbitrary mix of parts, is unlikely to be achievable, but limited versions are.

V. FORMAL MODELS

This section describes the way we model the software view of the factory floor and process plans. We need these models to develop control algorithms. Since we realize the factory floor and process plans as assemblages of software/hardware components, we are, in effect, concerned with formal semantic models for such components. The modeling methodology used is described in more detail in [21]. It is relatively simple formalism that is just expressive enough to capture the level of detail—the logical level—with which we are concerned, yet avoids a modeling methodology that is too complicated to be practical. Further, it is a structure that lends itself well to application of artificial intelligence techniques.

some robots, NC machines, and inspection devices. The information passed to it through generic parameters would specify how many and which kinds of each are in the specific cell. That is, it is given information about the components with which it will interact, and these, in turn, might be the result of earlier instantiations of generic components.

If we continue in this manner assembling every more elaborate components out of components and instantiations of generic components, we could arrive at a kind of generic component that is of central interest to us: generic factory control components. A given generic factory control component would be capable of controlling any one of a large class of factory floors making any one of a large class of product mixes. The information passed through

A. Formal Modeling Concept

Consider a simple a manufacturing cell, i.e., a software/hardware component, whose semantics we want to model, one made up of a robot, an NC machine, a load/unload port, and parts that pass through the system. A simplified interface might be as follows:

```
package MANUFACTURING_CELL is
  procedure LOAD_PART_ONTO_MACHINE;
  procedure UNLOAD_PART_FROM_MACHINE;
  procedure MACHINE_PART;
end MANUFACTURING_CELL.
```

The procedure names and arguments (if present) characterize inputs and outputs. The semantics of the cell correspond to the body of the component MANUFACTURING_CELL, and it is these that we want to model.

We make a distinction between two aspects of manufacturing systems, calling them, respectively, the Euclidean and logical levels. The Euclidean level is characterized by the locations, orientations, and movements of objects expressed in terms of finite dimensional Euclidean spaces. Models are typically coordinate transformations, kinematic constraint equations, differential equations, etc. The logical level is an abstraction of the Euclidean level and is concerned with logical conditions, transformations of logical conditions, and events. Variables simply identify things and, with the exception of time, are not real valued, e.g., a condition might be that "the material transport system is at machine number seven."

Obviously, there are important control software problems at both the Euclidean and logical levels. However, the Euclidean control issues are usually local, for example, NC programs and robot programs. The manufacturing system modeling addressed in this paper is primarily concerned with control software for the logical level, since the larger integration issues arise at this level. Still, the software components framework of Section III is equally valid in both cases.

Our goal, then, is to model the logical view of components such as MANUFACTURING_CELL. Central to our modeling system is the concept of system state. Roughly speaking, we take the state to be a composite of 1) the set of entities that make up the manufacturing system, 2) the classification of each entity (e.g., robot, part), and 3) "where" the entities are with respect to one another. We say "where" in quotes because, at the level of abstraction being addressed by this model, it is only necessary to know that the part is on the machine without giving the coordinates of the machine and/or the parts.

More precisely, the state of the model at any given time is characterized by a structure [13] for a first-order language. This language always has a one-place predicate symbol, AE , and two two-place predicate symbols, CT and ST . In addition, it has a number of other, usually one-place, predicate symbols $V_\alpha, V_\beta, \dots$, and it has constant symbols.

A structure characterizing the state is referred to here as a configuration, and it has

- a universe set called E_u ,
- a subset of E_u associated with the predicate symbol AE . (We also denote this set by AE and rely on context to clarify whether the predicate symbol AE or the set AE is meant. We will treat all predicate symbols and their associated relations in the same manner.)
- subsets CT and ST of $E_u \times E_u$ associated with the predicate symbols CT and ST , respectively,
- subsets $V_\alpha, V_\beta, \dots$ of the appropriate Cartesian product of E_u with itself associated with the predicate symbols $V_\alpha, V_\beta, \dots$,
- assignments of each constant symbol to an element of E_u .

The elements of E_u are called entities, and they are usually things such as parts, assemblies, tools, machines, and messages. That is, they usually have a direct and obvious connection to the real world. E_u is meant to be the set of all entities that will ever be of interest in the model. Those entities that are currently active are in the set AE , for example, the parts currently on the factory floor. The sets $V_\alpha, V_\beta, \dots$ are value sets. For example, $x \in V_{part_i}$ or, equivalently, $V_{part_i}(x)$ designates that the entity x is a part of type i . The entities that are active may change with time; however, the values of an entity are fixed forever. That is, a part is always a part. Similarly, the element of E_u assigned to a particular constant symbol never changes.

$CT \subset E_u \times E_u$ is the contact relation, and it is used to model such things as the part being on the machine, that is, "the part and the machine in contact." $ST \subset E_u \times E_u$ is the substructure relation, and it is used to model such things as the part being a substructure of the assembly.

A configuration, then, is a structure making the form of the state of the system precise. However, we know that some portions of the structure never change—the values, the assignments of constants, and the universal set E_u —so these can be suppressed in the description of the state, and we can simply keep track of the ordered-triple (AE, CT, ST) .

In the simple component MANUFACTURING_CELL, given at the beginning of this section, the semantics might⁴ be as follows: the entity set E_u has entities MACHINE, PORT, MACHINING, and an arbitrary number of parts. The PORT is the place where parts enter and leave the cell. MACHINING would signal that machining was in progress. AE at a given moment would always contain MACHINE and PORT, for we assume they are always active. AE might at the same time contain MACHINING and some number of parts. CT would show where the parts are (i.e., in contact with MACHINE or PORT) and if MACHINE is machining (i.e.,

⁴We say "might" because it is unrealistic to expect the semantics of a component to be determined by its public interface.

MACHINING is contact with MACHINE). Since the example is so simple, there is no need to use the substructure relation ST .

The operation of the system, then, becomes a sequence (AE_k, CT_k, ST_k) of configurations or states. We now describe how state transitions are modeled.

The most basic thing that can happen is a *change*, and, as we will see, the semantics of the logical view of a software component are modeled with a collection of changes. A change is represented as

$$L(t) \mapsto R(t_1(t))$$

where L and R are sets of well-formed formulas. The formulas in L refer to the configuration at the start of the change, and those in R refer to the configuration after the change is finished. If all the formulas in L are satisfied, then R , which is usually not satisfied by the current configuration, becomes satisfied by the new configuration. If the change starts at time t , then it finishes at $t_1(t)$, for example, $t_1(t) = t + 12$.

To illustrate the use of changes, let us formulate a semantic model of the logical view of the procedure MACHINE_PART in the software component MANUFACTURING_CELL. The other procedures would be modeled in a similar manner, and taken together the models of the three procedures would be a model of the software component. Needless to say, there are numerous ways that procedure MACHINE_PART might work. We will pick one of them. We suppose that if there is a part on the machine and no machining is taking place, then calling the procedure starts machining and a message to that effect is sent. Furthermore, machining is completed 12 minutes later and a message to that effect is sent. If the procedure is called in any other case, it does nothing. The following three changes comprise the formal semantic model:

$$\begin{aligned} & \{ I_{\text{MACHINE_PART}} \} (t) \mapsto \{ \neg I_{\text{MACHINE_PART}} \} (t) \\ & \left\{ \begin{array}{l} I_{\text{MACHINE_PART}} \\ (\exists x)(CT(x, \text{MACHINE}) \wedge V_{\text{PART}}(x)), \\ \neg CT(\text{MACHINING}, \text{MACHINE}) \end{array} \right\} (t) \\ & \mapsto \left\{ \begin{array}{l} CT(\text{MACHINING}, \text{MACHINE}), \\ I_{\text{SEND_MACHINING_STARTED_MESSAGE}} \end{array} \right\} (t) \\ & \left\{ \begin{array}{l} I_{\text{MACHINE_PART}} \\ (\exists x)(CT(x, \text{MACHINE}) \wedge V_{\text{PART}}(x)), \\ \neg CT(\text{MACHINING}, \text{MACHINE}) \end{array} \right\} (t) \\ & \mapsto \left\{ \begin{array}{l} \neg CT(\text{MACHINING}, \text{MACHINE}) \\ I_{\text{SEND_MACHINING_FINISHED_MESSAGE}} \end{array} \right\} (t + 12). \end{aligned}$$

In effect, the logical variable $I_{\text{MACHINE_PART}}$ is an input to the formal model which may be set to true from outside the model. Setting it to true models calling the procedure MACHINE_PART. The first change is really a technicality; it

shows that if $I_{\text{MACHINE_PART}}$ is set to true, then it is immediately reset to false from the inside the system. The point, of course, is that calling the procedure is a discrete event.

The left side of the second change shows that to start machining there must be a part on the machine, that is, $(\exists x)(CT(x, \text{MACHINE}) \wedge V_{\text{PART}}(x))$, and machining must not already be in progress, that is, $\neg CT(\text{MACHINING}, \text{MACHINE})$. In the right side, machining is started, that is, $CT(\text{MACHINING}, \text{MACHINE})$, and a message is sent, that is, $I_{\text{SEND_MACHINING_STARTED_MESSAGE}}$. Presumably, setting the logical variable $I_{\text{SEND_MACHINING_STARTED_MESSAGE}}$ corresponds to calling some other procedure. The times are the same on both sides of the change to indicate that the change is essentially instantaneous. Note that if $I_{\text{MACHINE_PART}}$ is set to true and the rest of the left side is not satisfied, then nothing happens just as we described in words.

The left side of the third change is exactly the same as the second change, so this change occurs if and only if the other one does. The configuration changes to satisfy the right side 12 units of time after machining starts, and it shows that machining has completed, that is, $\neg CT(\text{MACHINING}, \text{MACHINE})$. It also sends a message to this effect.

Presumably, there will be constraints on the simultaneous execution of the procedures in MANUFACTURING_CELL. We cannot load and unload at the same time. What happens when several of the procedures are called at essentially the same time? Clearly, there is no simple answer. It depends on the way that the real manufacturing cell works, how many processors it has, and so forth; and other software components can be considerably more complicated than our simple cell. However, here it is reasonable to assume that no two changes may overlap in time. Further, if, for example, it was impossible at the time of a call to unload the machine, the procedure might do nothing or it might send a message saying that unloading was currently impossible. The formalism can model these cases as well. It can also model another likely way that the procedures might work. It could be that the body of MANUFACTURING_CELL effectively queued up calls to the various procedures as is done, for example, at entries to Ada tasks. The formalism can express this also. Thus it is possible to model highly constrained situations, sequential ones, and highly parallel ones.

The formalism has another important advantage: it blends well with the assembly of software/hardware components. By this we mean that creating a semantic model for an assemblage from the semantic models for the constituent components is relatively easy. Although there are several ways to do this [21], the most useful so far has been based on a combination of "unions" and "views." Roughly speaking, the union of two or more models is, as the name suggests, obtained by taking the union of configurations and the union of the sets of changes. For example, if (AE_1, CT_1, ST_1) and (AE_2, CT_2, ST_2) are configurations, their union is $(AE_1 \cup AE_2, CT_1 \cup CT_2, ST_1 \cup ST_2)$; and if C_1 and C_2 are the set of changes for each constituent

component, the set of changes for the union is $C_1 \cup C_2$. Consider two very simple systems: one has a machine, a loading place, and parts; the other system has the loading place, a vehicle, and parts. The changes in the first allow parts to be moved back and forth between the machine and the loading place. The second allows parts to be moved back and forth between the loading place and the vehicle. In the union parts can be moved from the machine to the vehicle by way of the loading place and vice versa. However, note that the functionality of the union is essentially the union of the functionalities of the constituent components. Since this is often not true of an assemblage, we need to go beyond unions.

A view shows part of a system; in particular, it shows part of the configuration and parts of some of the changes. In other words, a view exhibits some of the functionality of the viewed system. For example, in the union just discussed, add two new changes: one moves a part from the machine to the vehicle, and the other moves a part from the vehicle to the machine. These additions create a new system. Although neither added change is a change in the union, each can obviously be defined in terms of changes that are in the union. In any event, the union is obviously a view of this new system, since the new system is just the union plus the two added changes. Another, more interesting, view of the new system shows only the machine, vehicle, a part, if one is present, and the two new changes. The loading place is not in the view, nor are the changes in the union. In effect, this view is an assemblage of the two original systems, and its functionality is defined in terms of their functionalities; however, it hides, as most assemblages do, details of the original systems. This example is too simple to show it well, but usually the functionality of such a view will be a restriction on the functionality of the union of the original systems, that is, complete access to the union might allow something that is not possible through the view. This also is the way we expect assemblages of hardware/software components to work.

Another important use of the above formalism is to model process plans. In particular, it is crucial for flexibility that the model of the factory and the models of process plans be separate. One wants to be able to reuse all these models; and if they were intertwined, reuse would be impossible. In a specific application one simply combines (i.e., union) these models to get a model for the application. Thus it is important to model factories and process in the same way. For example, a process plan for a part in a more elaborate manufacturing system than MANUFACTURING_SYSTEM might have five steps, each one to be performed on a particular kind of machine. In a formal model each step would be associated with an entity of the type STEP. The changes of this model would enforce the required ordering of these steps. Thus a condition for starting the second step in the process plan would be that the first step is finished, and so on. Part entities would move through the process plan, moving from step to step. In this way the process plan model could keep track of any number of parts as they progress through the process plan.

It would guarantee that the ordering constraints of the process plan were satisfied. However, the process plan model would not show the constraints associated with moving the parts on the factory floor. For example, process plans do not mention material transport systems. We need a formal model of the factory floor, one of the type we have been discussing. Thus a factory floor model shows the constraints associated with competition for, say, machines and transport, and the process plan model shows steps and their required ordering. Since we must be concerned with both, these two models must be combined with unions and views. Finally, if more than one kind of part is being manufactured, there will be more than one kind of process plan model in the combination.

B. Relation to Other Semantic Models

Our method of modeling software components is reminiscent of formal methods used to model abstract data types and the semantics of programs, but there are differences. The most obvious ones are the explicit role of time and the possibility of genuine parallelism. In other methods, time is rarely part of the model, and parallelism, when considered, is usually treated in terms of concurrency. We can ignore neither.

Another obvious difference is the potentially active nature of the software components. Usually formal models of abstract data types implicitly assume a passive object. Such an assumption is not tenable when one is modeling a manufacturing cell with random delays and spontaneous events, and our formalism does not make such an assumption. Thus, our concept of a software component—the active software/hardware component—is a generalization of the usual formal characterization.

Our approach is also similar to operational semantics [22, pp. 19–21, 337] in the sense that we seem to define an abstract machine. However, there is a difference. Here state transitions are defined syntactically in terms of sets of well-formed formulas. This gives rise to a problem that we call the “updating problem.” It arises because the configurations or states for some syntactically defined changes are ambiguous. The problem is essentially the world selection problem in counterfactual reasoning [23], [24]. The problem has been solved in [25] in the sense that simple necessary and sufficient conditions have been developed that identify unambiguous changes.

A similarity to axiomatic semantics [26], [27] also exists in that we have “before” and “after” conditions. However, the before and after conditions of axiomatic semantics are usually not intended to define the program fragment, while this is precisely our intention: we have no other way to say what a change does.

A connection to denotational semantics [22], [28] is also possible, but in our case the emphasis on how a configuration is reached is so paramount that denotation of the reached configuration is really secondary. Moreover, our need to allow genuine parallelism seriously limits the usefulness of the process model of program semantics [29], [30].

Our approach is also related to the algebraic approach to modeling and specifying abstract data types [10, pp. 189–218], [26], but again there are differences—those already discussed plus another one: it is awkward to treat the software/hardware components that we are interested in as algebras. In particular, before and after conditions are conditions on entities and pairs of entities, whereas changes (i.e., operators) alter the relations *AE*, *CT*, and *ST*. This means that there is an implicit second-order logic present, and this is the source of the awkwardness.

For example, the configurations in our formal model can be viewed as relational databases, and this would probably be the way that they would be recorded. Changes, then, are in part operations on databases; in fact, they are updating operations. However, note that this means that we need two kinds of variables: entity-variables for things such as parts and machines, and the “database”-variables *AE*, *CT*, and *ST*. The latter are predicate-variables, and this, in turn, means that we have a second-order logic [13, p. 269].

Yau and Caglayan [32] model software components using modified Petri nets, and many of the differences already discussed apply here. However, they do develop formal methods for interconnecting software components: they “tie” two Petri nets (i.e., components) together.

Finally, it is obvious that our modeling approach is similar to the situation-based models used in planning [33]–[35]. There are differences; however, for the purposes of this article they can be ignored.

C. Relation to Manufacturing System Models

Many modeling techniques for manufacturing systems exist [36], [37]; however, most are not appropriate for describing, as we want to do, a factory floor and associated process plans to a generic control component. For example, industrial engineers have used networks of queues [38]–[41] and Markov processes [42]–[44] extensively to model manufacturing systems.

In queuing networks control is implicit in the structure of the model, a number of details of the logical level are suppressed, and the process plan models are also implicit in the structure of the model. Such models are also inappropriate for our purposes in that their product is equilibrium statistics, and these are of indirect and limited benefit when trying to develop real-time controls. In other words, these models are at too high a level and yield, for our purposes, the wrong perspective on the manufacturing system. Markov models have essentially the same kinds of problems with the one exception that control can be made explicit.

Petri net models of manufacturing systems [45], [46] remove several of these difficulties. They can model at a level of detail appropriate for treating control of the logical level, and generalizations allow time and stochastic behavior to be modeled. However, our approach appears to lead to more compact models and be more natural for the modeler. Particularly important is the fact that the connection between changes and software component in-

terfaces of our modeling system yields a natural bridge between software and models. Further, ours is based on a branch of mathematics, mathematical logic, that has decades of developments to call upon, and is in a form that allows rather direct use of artificial intelligence and relational database ideas. Still, a careful comparison of our formal models and Petri net models needs to be made. For example, it appears that any Petri net model that does not contain Euclidean level information can be modeled more concisely using our formalism. More importantly, it appears that our modeling system is more expressive in that it can model behavior that either cannot be modeled or is awkwardly modeled using even colored Petri nets [47]. On the other hand, there are Petri net concepts such as invariants and linear system theory ideas based on [48] that might be modified to be suited to our models.

Manufacturing systems are also being modeled with rule-based systems [49], and one interpretation is that ours is a rule-based model. However, there is a subtle difference. Simple rule-based models—that is, ones without truth maintenance systems—do not have an updating problem because their rules state precisely what has to be added to the database. In our case a *change* expresses a condition that the updated database must satisfy but does not explicitly specify what the next database should be. That is, in our case specifying a new fact often carries with it the need to change old facts. In other words, we need the truth maintenance system that is a solution to the updating problem discussed earlier. Since this truth maintenance system is independent of the structure of a particular model, it is not part of the model and remains implicit. It is precisely this implicitness that makes our models concise. Finally, note again that we allow true parallelism, and this is not typical of rule-based systems.

Although automatic or generative process planning has been investigated, little attention has been given to modeling process plans in a manner that allows models of the manufacturing system and models of the process plans to be combined into a common formal framework while maintaining their separate identities. This is an important feature of our approach. We view the process plans as “programs” for the factory floor “computer,” and it is even sometimes useful to think of the generic factory control system as a kind of operating system that sees to the fair and timely execution of these “programs.” An interesting advantage is that this “operating system” knows a great deal about the future in that it can look ahead through all the active process plans. Naturally, there is still uncertainty: machines can break down, new orders can arrive or be cancelled, etc. Still, this operating system has far more knowledge of the future than real operating systems do.

VI. DISTRIBUTED LANGUAGE ENVIRONMENTS

In this section we discuss the programming language aspects of our conceptual framework. In particular, we are concerned with a distributed language environment that

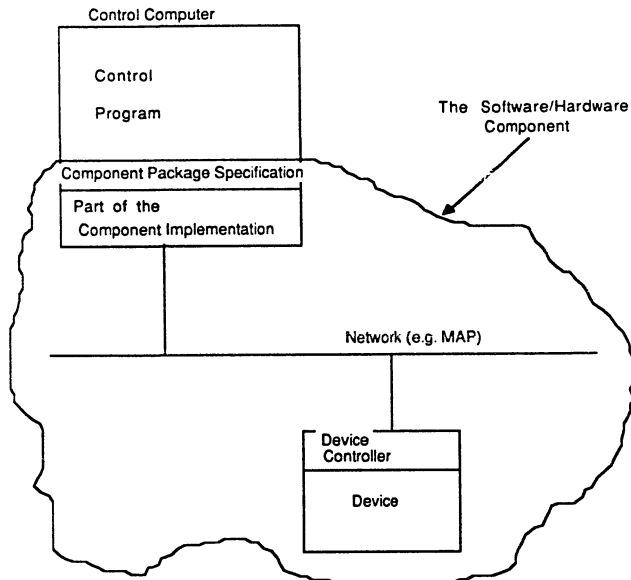


Fig. 8. Software component requiring distributed execution.

supports real-time software/hardware components, their assemblages, and generic components.

A. Distributed Computing for the Conceptual Framework

To illustrate the role of distributed program execution in our framework, consider the situation in Fig. 8. In this case, the control software resides on one computer, requiring that the specification of the software/hardware component also be available on that computer. The body implementing the component is split between that and another computer. The principal issue is that the logical function performed extends across more than one processing unit. To us, then, a distributed language environment means more than just writing programs in the same language on each machine in the system. It means distributed execution of a single program across machine boundaries. In this way the advantages of proven software engineering mechanisms⁵ can be achieved in the distributed environment. These capabilities are accepted as being essential for the effective development of large complex programs in a uniprocessing environment [50]–[52]. They can certainly be no less important in the distributed environment. Indeed, a distributed language capability is fundamental to our approach to manufacturing software.

B. Distributed Language System

A number of distributed languages have been proposed, e.g., [16], [18], [19], [53]–[66]. However, most were intended primarily for studying such things as synchronization and communication primitives. Only a few languages, e.g., [16], [57], [60], seriously consider real-time issues, most do not

⁵These mechanisms include such things as 1) data encapsulation and hiding, 2) abstract data types, 3) modularization of programs, 4) separate compilation (of both modules and specifications), 5) concurrency mechanisms at the language level, and 6) extensive compile time error checking.

provide generic capabilities, and there are many aspects of distributed execution they do not consider. There is certainly no available widespread vendor-implemented, distributed language, and one will eventually be needed to support our conceptual framework.

Our conceptual framework must operate in real-time as well as on distributed processors. It is one thing to create powerful expressive mechanisms for distributed program representation; it can be quite another to implement them efficiently enough for real-time operation. For example, MAP is a reasonably logical approach to communication, but it is now suspected that the cost of its elegance is slowness, e.g., it takes up to 100 ms. for an end-to-end message transmission, regardless of network speed [11]. We must be careful not to create a comparable problem.

Our approach to this need has been to adopt a standard programming language intended for real-time operation and develop a distributed version of it. Because it is basically a good language, is subject to intense standardization efforts, and is ostensibly intended for distributed execution, we selected Ada. To achieve distributed execution, we have built a pretranslator that takes a single Ada program as an input and whose output is a collection of pure Ada programs, one for each targeted processor. This is somewhat akin to the way embedded SEQUEL is handled in the DB2 database management system. This approach illustrates two key views we hold.

First, we do not believe that a special-purpose distributed language should be developed for integrated manufacturing systems. This, we believe, would be a dead end, and guarantee that manufacturing software would remain an expensive cottage industry. Rather, we believe that our conceptual framework—and manufacturing software in general—should, as much as possible, be in the mainstream of software development and utilize the extensibility of modern languages, proposing extensions only where absolutely necessary.

Second, as we have said earlier, we believe that control software should be written as one program in a common distributed language. Our interim approach allows this, and we have experienced some of the advantages discussed earlier, even in this initial version.

Our distributed Ada system allows us to distribute library packages and library subprograms statically among a set of homogeneous processors. We write a single program and use a pragma (essentially a compiler directive) called SITE to specify the location on which each library unit is to execute. For example, if the simple transport system mentioned earlier were controlled by computer 2 and the cell control using it were on computer 1, a sample of relevant code might look as follows:

```
pragma SITE (2);
package VEHICLE is
  procedure MOVE_FORWARD;
  :
end VEHICLE;
:
```

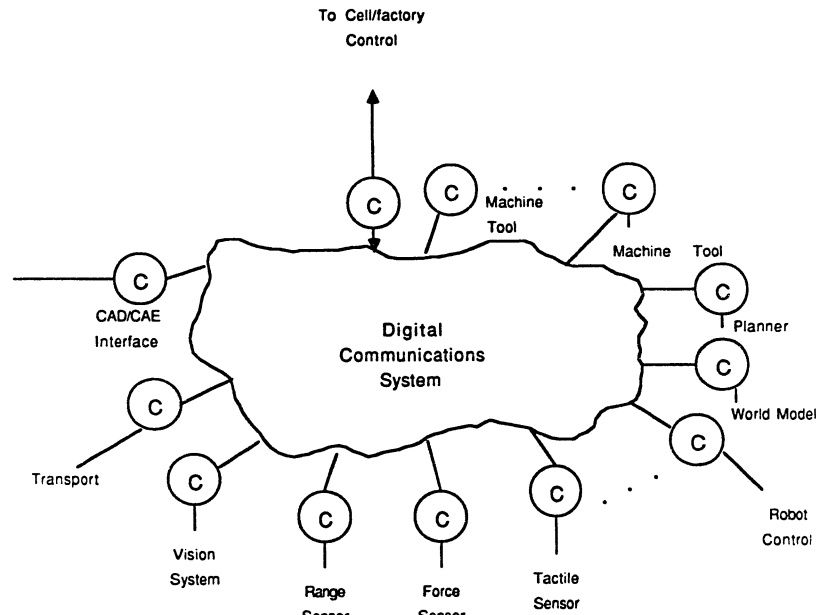


Fig. 9.

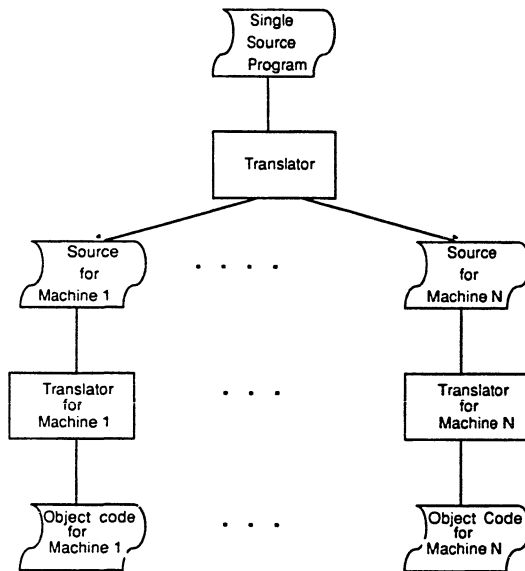


Fig. 10.

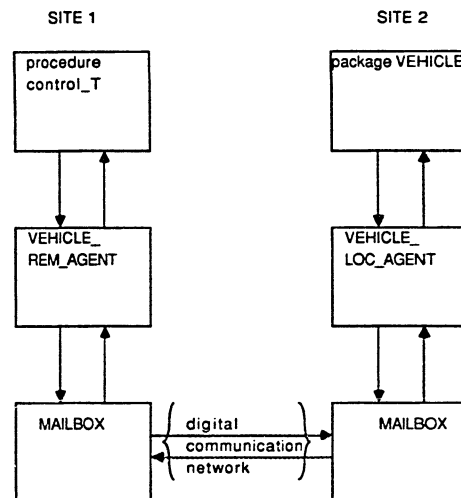


Fig. 11.

```

pragma SITE (1);
with VEHICLE;
procedure CONTROL is
  .
  .
begin
  .
  VEHICLE.MOVE_FORWARD;
  .
end;
  
```

Our translation system would replace the local call to the procedure `VEHICLE.MOVE_FORWARD` with the

appropriate remote call. Similarly, any references in `CONTROL` to data objects defined in package `VEHICLE` would be translated into appropriate remote references as would task entry calls.

The operation of our translator is illustrated in Figs. 9–11. We presume that our computers are interconnected by a communication network, as shown in Fig. 9. Fig. 10 illustrates the operation of our translator. It translates our single Ada program into a set of independent Ada programs that include library modules of our design to effect communication among processes. Each of the individual programs can thus be compiled by an existing Ada com-

piller. This approach simplifies the translation process considerably since our pretranslator is much less complex than a full Ada compiler.

The communication is managed by a set of agents [67] created for units that can be remotely referenced and an underlying process-to-process mailbox routine. Roughly speaking, the relevant entities and flow of data are as shown in Fig. 11. CONTROL_T is the translation of CONTROL with the references to objects in VEHICLE replaced with calls into VEHICLE_REM_AGENT.

The operation of this system can be modeled in terms of the run time overhead associated with various kinds of remote references. From the tests performed in [68] we know that task rendezvous times exceed procedure call times by one and a half to two orders of magnitude. We can also reasonably expect the network communication times to be sizable. For example, message end-to-end times for MAP are on the order of 100 ms, [69], for the Intel hypercube, a few milliseconds, and for the NCUBE hypercube, several hundred microseconds to a millisecond. Thus we neglect all local procedure and function call times and model our overhead in terms of the number of messages and local rendezvous required. Let t_m and t_r be the times to complete a message transfer and local rendezvous, respectively, and let n_m^o and n_r^o be the number of messages and local rendezvous required for a remote operation of type o . Then, the time to complete a remote operation is

$$n_m^o \cdot T_m + n_r^o \cdot t_r.$$

For task rendezvous we have $n_m^r = 2$, $n_r^r = 6$ and the remote rendezvous time is $2 \cdot t_m + 6 \cdot t_r$. For example, if we have a slow network in which $t_m = 100$ ms and an Ada compiler that produces 0.500 ms rendezvous times (several such compilers now exist for Motorola 680X class processors), we would have a remote rendezvous time of 203 ms, obviously, heavily dominated by the message passing time. For a fast network with, say, a t_m of 500 μ s, we would have a rendezvous time of 4 ms, which is close to single machine rendezvous times of a year or two ago [68].

Clearly, within the message passing times achievable with today's technology, a reasonable distributed language capability is feasible. For a more complete description of the distributed Ada translator system, see [67].

VII. EXAMPLE

A preliminary version of a generic factory floor controller using simulation instead of real components is operational on a pair of VAX's. The control component is on one VAX and a simulation of the factory floor on the other, as shown in Fig. 12. The factory floor controller sends commands to the factory floor simulator on VAX 1. Each time something changes in the simulator the controller is notified, and the calls for the same change in the factory floor tracking model in VAX 1. Except for a small time delay, the configuration of the tracking model is supposed to be the same as that of the factory floor

simulator. The controller uses the process plan tracking model to keep track of where parts are in their process plans. There is no comparable model on VAX 1 because a model of the factory floor says and knows nothing about process plans. The connection between movement on the factory floor and progress through the process plan must be made on VAX 2. The search models on VAX 2 are used to select the next commands to send to the factory floor simulator. At the beginning of a command-selection cycle the configuration of the search model is made the same as that of the tracking model. Then, the controller uses the search model to build a tree of various future scenarios. A few simple pruning rules keep the search manageable. The most desirable path through the tree is selected, and its initial part is used to select the next command(s). This next command(s) is sent to the factory floor, and the cycle begins again.

Carefully note that factory floor models, process plan models, and the controller are separate in this system. In principle, but not completely in practice, one should be able to change any one of them. This is a major source of flexibility.

Also note that communication is in dashed boxes, that is, the agents. This is done to indicate that communication is invisible to the programmer. In particular, the FACTORY_FLOOR component on VAX 1 "thinks" that it is connected directly to the FACTORY_FLOOR_CONTROLLER on VAX 2 and vice versa. Our approach to a distributed language, discussed in Section VI, is that the programmer should not have to be concerned with this communication software or the extra specifications. This is the job of the language translation system.

VIII. CONCLUSION

A coherent approach to manufacturing software is one of the most important building blocks needed for US industry to develop integrated manufacturing systems. We have described a concept by which coherent manufacturing can be accomplished. However, the theory is not yet complete. Indeed, much remains to be done. Extensions to the formal modeling system are needed to more fully handle generics and distribution of components. The process of instantiation of generics to real components must be extended to allow dynamic instantiations. Distributed languages must be studied in a more general context of multiple forms of memory interconnections, multiple possible binding times, and various degrees of homogeneity (e.g., see the major dimensions of a distributed language defined in [70]).

Yet, we have accomplished enough to demonstrate the viability of the major underlying ideas. A primitive version of a distributed Ada translation system is working, and a limited generic real-time factory controller is operational, with real factory components replaced by simulation. We believe that when it is fully developed, the approach presented here can become the heart of future integrated manufacturing systems.

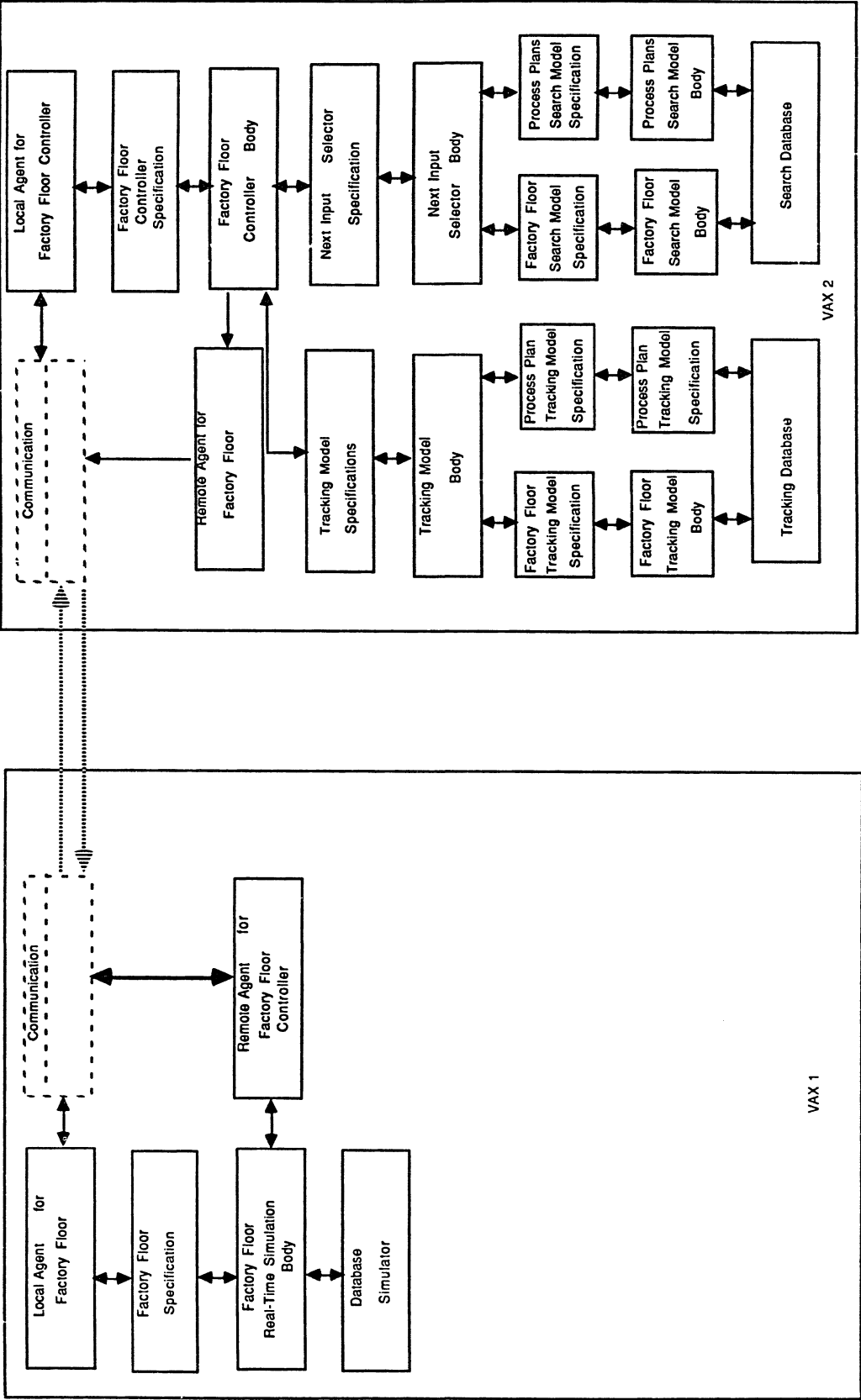


Fig. 12.

REFERENCES

- [1] R. A. Volz, T. N. Mudge, A. W. Naylor, and B. Brosgol, "Ada in a manufacturing environment," in *Proc. 5th Annu. Control Eng. Conf.*, Rosemont, IL, May 1986, pp. 433-440.
- [2] Y. Koren, *Computer Control of Manufacturing Systems*. New York: McGraw-Hill, 1983.
- [3] *User's Guide to Val, Version II*, 2nd ed., Unimation Inc., Sept. 1982.
- [4] *IBM Robot System/1 AML Reference Manual*. IBM Corp., Boca Raton, FL, 33432, 1981.
- [5] S. Bonner and K. G. Shin, "A comparative study of robot languages," *IEEE Computer*, pp. 82-96, Dec. 1982.
- [6] M. A. Kaminski, "Protocols for communicating in the factory," *IEEE Spectrum*, vol. 23, pp. 56-62, Apr. 1986.
- [7] J. A. Simpson, R. J. Hocken, and J. S. Albus, "The automated manufacturing research facility of the national bureau of standards," *J. Manuf. Syst.*, vol. 1, pp. 17-32, 1982.
- [8] D. Maier, *The Theory of Relational Databases*. Computers Science Press, 1983.
- [9] A. S. Tanenbaum, *Computers Networks*. Englewood Cliffs, NJ: Prentice-Hall, 1981.
- [10] J. C. Cleaveland, *An Introduction to Data Types*. Reading, MA: Addison-Wesley, 1986.
- [11] R. A. Volz and A. W. Naylor, "Final report of the NSF Workshop on Manufacturing Systems Integration," Robotic Syst. Div., Center for Research on Integrated Manufacturing, College of Engineering, Univ. of Michigan, Ann Arbor, Tech. Rep. RSD-TR-17-86, Nov. 1985.
- [12] R. A. Volz and T. N. Mudge, "Robots are (nothing more than) abstract data types," in *Proc. SME Conf. Robotics Research: The Next 5 Years and Beyond*, Aug. 14-16, 1984, pp. MS84-493: 1-16.
- [13] H. B. Enderton, *A Mathematical Introduction to Logic*. New York: Academic, 1972.
- [14] J. G. P. Barnes, *Programming in Ada*, 2nd ed. London: Addison-Wesley, 1984.
- [15] N. Wirth, *Programming in Modula-2*. New York: Springer-Verlag, 1982.
- [16] *Ada Programming Language (ANSI/MIL-STD-1815A)*, Ada Joint Program Office, Dep. of Defense, OUSD(R&D), Washington, DC, Jan. 1983.
- [17] B. Stroustrup, "An overview of C++," *Assoc. Comput. Mach. Sigplan Notices*, vol. 21, pp. 7-18, Oct. 1986.
- [18] R. E. Strom and S. Yemini, "Nil: An integrated language and system for distributed programming," in *Proc. Sigplan '83 Symp. Programming Language Issues in Software Systems*, June 1983, pp. 73-82.
- [19] C.A.R. Hoare, "Communicating sequential processes," *Commun. Assoc. Comput. Mach.*, vol. 21, pp. 666-677, Aug. 1978.
- [20] C. Antonelli and R. A. Volz, "Hierarchical decomposition and simulation of manufacturing cells using Ada," *Simulation*, vol. 46, Apr. 1986.
- [21] A. W. Naylor and M. C. Maletz, "The manufacturing game: a formal approach to manufacturing software," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-16, pp. 321-334, May/June 1986.
- [22] J. E. Stoy, *Denotational Semantics: The Scott-Strachey Approach to Programming Language Theory*. Cambridge, MA: MIT Press, 1977.
- [23] D. K. Lewis, *Counterfactuals*. Cambridge, MA: Harvard Univ. Press, 1973.
- [24] W. L. Harper, R. Stalnaker, and G. Pearce, eds., *IFS Conditionals, Belief, Decision, Chance, and Time*. Dordrecht, The Netherlands: R. Reidel, 1981.
- [25] A. W. Naylor and M. B. Shadmehr, "A geometric approach to counterfactual reasoning and its application to modeling manufacturing systems," Robotic Syst. Div., Center for Research on Integrated Manufacturing, College of Engineers, Univ. of Michigan, Ann Arbor, Tech. Rep. RSD-TR-5-87, Mar. 1987.
- [26] R. W. Floyd, "Assigning meanings to programs," *Proc. of Symposia in App. Math.*, vol. 19, pp. 19-32, 1967.
- [27] C. A. R. Hoare, "An axiomatic basis for computer programming," *Comm. Ass. Comput. Mach.*, vol. 12, pp. 576-580, 1969.
- [28] M. J. C. Gordon, *The Denotational Description of Programming Languages*. New York: Springer-Verlag, 1979.
- [29] R. E. Milne, "The formal semantics of computer languages and their implementations," Ph.D. dissertation, Univ. of Cambridge, 1974.
- [30] A. J. R. G. Milner, "An approach to the semantics of parallel programs," in *Proc. Convegno di Informazione Teorica*, Istituto Di Elaborazione della Informazione, Pisa, 1973.
- [31] J. A. Goguen and J. W. Thatcher, "Initial algebra semantics," in *Proc. 15th IEEE Symp. Switching and Automation Theory*, New Orleans, Oct. 1974.
- [32] S. S. Yau and M. U. Caglayan, "Distributed software system design representation using modified petri nets," *IEEE Trans. Soft. Eng.*, vol. SE-9, pp. 733-745, Nov. 1983.
- [33] E. Rich, *Artificial Intelligence*. New York: McGraw-Hill, 1966.
- [34] C. Green, "Theorem proving resolution as a basis for question-answering system," *Machine Intell.*, 1969.
- [35] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial Intell.*, 1971.
- [36] *Proc. 1st ORSA/TIMS Spec. Int. Conf. Flexible Manufacturing Systems*, Oper. Res. Soc. Amer. and The Inst. of Manag. Sci., Aug. 1984.
- [37] R. Suri, "An overview of evaluative models for flexible manufacturing system," in *Proc. First ORSA/TIMS Conf. FMS*, Aug. 1984.
- [38] J. A. Buzacott and J. G. Shanthikumar, "Models for understanding flexible manufacturing systems," *Trans. AIEE*, vol. 12, pp. 339-350, Dec. 1980.
- [39] J. Solberg, "Analytic performance evaluation for the design of flexible manufacturing systems," in *Proc. Conf. Decision and Control*, 1979.
- [40] C. Whitney, "Control problems in flexible manufacturing," in *Proc. 1984 Conf. Decision and Control*, San Diego, 1984.
- [41] R. Suri and R. R. Hildebrandt, "Modelling flexible manufacturing systems using mean value analysis," *J. Manuf. Syst.*, vol. 3, pp. 27-38, 1984.
- [42] S. Gershwin, R. Akella, and Y. Choong, "Short term production scheduling of an automated manufacturing facility," in *Proc. 23rd Conf. Decision and Control*, Dec. 1984.
- [43] S. Gershwin, "Manufacturing systems modeling and control," in *Proc. IEEE Conf. on Robotics and Automation*, Apr. 1986.
- [44] J. Kimemia and S. Gershwin, "An algorithm for the computer control of a flexible manufacturing system," *IEE Trans.*, vol. 15, pp. 353-362, Dec. 1983.
- [45] C. L. Beck, "Modeling and simulation of flexible control structures for automated manufacturing systems," Robotics Inst., Carnegie-Mellon Univ., Pittsburgh, PA, Tech. Rep., 1985.
- [46] D. Dubois and K. E. Steckle, "Using petri nets to represent production processes," in *Proc. 22nd IEEE Conf. Decision and Control*, Dec. 1983, pp. 1062-1067.
- [47] M. Kamath and N. Viswanadham, "Applications of petri net based models in the modelling and analysis of flexible manufacturing systems," in *Proc. 1986 IEEE Int. Conf. Robotics and Automation*, Apr. 1986, pp. 312-317.
- [48] G. Cohen, D. Didier, J. P. Quadrat, and M. Viot, "A linear-system-theoretic view of discrete-event processes and its use for performance evaluation in manufacturing," *IEEE Trans. Automat. Contr.*, Mar. 1985.
- [49] M. S. Fox, B. P. Allen, S. F. Smith, and G. A. Strohm, "ISIS: A constraint-directed reasoning approach to job shop scheduling," Intelligent Systems Lab., Robotics Inst., Carnegie-Mellon Univ., Pittsburgh, PA, Tech. Rep. CMU-RI-TR-83-8, 1983.
- [50] E. Denert, "Software engineering: Experience and Convictions," in *Trends in Information Processing Systems. 3rd Conf. European Cooperation in Informatics*. New York: Springer-Verlag, 1981, pp. 16-35.
- [51] S. N. Woodfield, H. E. Dunsmore, and V. Y. Shen, "The effect of modularization and comments on program comprehension," in *5th Int. Conf. Software Engineering*, Mar. 1981, pp. 215-23.
- [52] L. Varga, "Specifications of reliable software," *Tanulmányok Magyar Tud. Akad. Számítástech. And Autom. Kut. Intez.* (Hungary), 113, pp. 309-25, 1980.
- [53] P. B. Hansen, *The Architecture of Concurrent Programs*. Englewood Cliffs, NJ: Prentice-Hall, 1977.
- [54] J. Van DenBos, R. Plasmeijer, and J. Stroet, "Process comm. based on input specifications," *Assoc. Comput. Mach. Trans. Programming Languages & Syst.*, vol. 3, pp. 224-250, July 1981.
- [55] G. R. Andrews, "Synchronizing resources," *Assoc. Comput. Mach.*

- Trans. Programming Languages & Syst.*, vol. 3, pp. 405-430, Oct. 1981.
- [56] J. A. Feldman, "High level programming for distributed computing," *Commun. Assoc. Comput. Mach.*, vol. 22, pp. 353-367, June 1979.
 - [57] R. Taylor and P. Wilson, "Process-oriented language meets demands of distributed processing," *Electronics*, vol. 55, pp. 89-95, Nov. 1982.
 - [58] M. Tsukamoto, *Language Structures and Management Method in a Distributed Real-Time Environment*. New York: Pergamon, 1981, pp. 103-130.
 - [59] W. V. Ruggiero and G. Bressan, "A programming model for distributed computing," in *Actas 2nd Conf. Int. Ciencia de la Computacion*, Aug. 1982, pp. 97-111.
 - [60] R. Holt, "A short introduction to concurrent Euclid," *Sigplan Not.*, vol. 17, pp. 60-79, May 1982.
 - [61] M. T. Liu and C.-M. Li, "Communicating distributed processes: A language concept for distributed programming in local area networks," in *Local Networks for Computer Communications, IFIP Working Group 6.4, Int. Workshop on Local Networks*, Aug. 1980, pp. 375-406.
 - [62] T. Christopher *et al.*, "Salad-a distributed compiler for distributed systems," in *Proc. 1981 Int. Conf. Parallel Processing*, Aug. 1981, pp. 50-57.
 - [63] P. E. Lauer and M. W. Shields, "Cosy-an environment for development and analysis of concurrent and distributed systems," in *Proc. Symp. Software Engineering Environments*, June 1980, pp. 119-156.
 - [64] P. B. Hansen, "Edison—A multiprocessor language," *Software — Prac. and Exper.*, vol. 11, pp. 325-361, Apr. 1981.
 - [65] R. P. Cook, "mod-A language for distributed programming," *IEEE Trans. Software Eng.*, vol. SE-6, pp. 563-71, Nov. 1980.
 - [66] T. W. Mao and R. T. Yeh, "Communication port: A language concept for concurrent programming," *IEEE Trans. Software Eng.*, vol. SE-6, pp. 194-204, Mar. 1980.
 - [67] R. A. Volz, P. Krishnan, and R. Theriault, "An approach to distributed execution of Ada programs," in *Proc. NASA Workshop on Telerobotics*, to appear 1987.
 - [68] R. M. Clapp, L. Duchesneau, R. A. Volz, T. N. Mudge, and T. Schultze, "Toward realtime performance benchmarks for Ada," *Commun. Assoc. Comput. Mach.*, pp. 760-778, Aug. 1986.
 - [69] T. N. Mudge, G. D. Buzzard, and T. S. Abdel-Rahman, "A high performance operating system for the NCUBE," *Proc. of the 1986 Conf. on Hypercube Multiprocessors*, Knoxville, TN, Oct. 1986.
 - [70] R. A. Volz, T. N. Mudge, G. D. Muzzard, and P. Krishnan, "Translation and execution of distributed Ada programs: Is it still Ada?" *IEEE Trans. Software, Special Issue on Ada*, to appear 1988.



Arch W. Naylor (S'51-A'52-SM'63) was born in Kansas City, MO. He received the B.S. degree from the University of California, Berkeley, in 1952, and the Ph.D. degree from the University of Michigan, Ann Arbor, in 1960, both in electrical engineering.

From 1952 to 1953 he worked on communication systems for the Collins Radio Company, Western Division, Burbank, CA. From 1953 to 1955 he was in the US Army Chemical Corps.

He was a Teaching Assistant in electrical engineering at the University of California, Berkeley, from 1955 to 1956. He was Fulbright Scholar at the Technological University, Delft, The Netherlands, for the academic year 1957 to 1958. From 1956 to 1960, except for the time in The Netherlands, he worked on nonlinear systems at the Cooley Electronics Laboratory, Department of Electrical Engineering, University of Michigan. During the academic year 1963 to 1964, he was a Fulbright Lecturer at the Technological University, Delft. He was a Professeur Associe at the University of Grenoble, France, during the academic year 1967 to 1968. He is currently a Professor of Electrical Engineering and Computer Science at the University of Michigan.



Richard A. Volz (M'60-SM'80) received the Ph.D. degree from Northwestern University, Evanston, IL, in 1964.

He has served as Associate Director of the University Computer Center and Associate Chairman of the Electrical and Computer Engineering Department and is currently Director of the Robot Systems Division of the Center for Research in Integrated Manufacturing. He joined the University of Michigan in 1964. He has worked on computational techniques for automatic control systems and done pioneering work on computer-aided design methods for control systems. Real-time distributed computing systems are a more recent technical interest of his, and he has led in the design and implementation of a real-time operating system, a higher level language for real-time control and a distributed language translator. His current research interests include distributed computer systems (particularly distributed languages), manufacturing software, embedded real-time computer systems, and robotics (particularly task planning, programming, and telerobotics). Particular projects include (CAD) model driven vision systems, automatic determination of gripping points on objects (from CAD information), distributed systems integration languages for real-time control, and telerobotics.